Localization

Please help improve this article or section by **expanding it** (https://eu5.paradoxwikis.com/index.php?title=Localization&action=edit) with: more details of data functions and other localization documentation.

Please help with verifying or updating older sections of this article.
At least some were last verified for version 1.0.

Localization files must be `.yml` format encoded in UTF-8-BOM. Otherwise, the game simply ignores the file. Further, the filename must end with `_l_<language>`; any file name can be used as long as this format is respected. For example: `MODNAME_USAGE_decisions_l_english.yml`.

The localization files are processed in reverse alphabetical order (from Z to A), adding `a` or `0` at the beginning of your localization file will make sure it applied last. When replacing vanilla localization, a better practice is to use a folder `replace` as in `/english/replace/overwriting_l_english.yml`. Files within a replace folder work slightly differently as the localization keys within are checked specifically and overwrite any other identical localization keys.

Note that unlike some previous Paradox Development Studio games, the game folders for displayed text are named "Localization" with a "z" rather than "Localisation" with an "s".

NOTE: All text presented to the player ***must*** be done through localization keys. Any "localized" text used in script types is actually just an unlocalized key that is presented verbatim. In certain script types – such as events, this can work, but other script types will show errors or not work as expected.

İçindekiler

Languages

Each supported language has an internal name which used as the name of the subfolder for the language. The internal name, with prefix `l_` (i.e. a lowercase L), is also used for the name format of the YAML files and as language identifier in the first line of the YAML file as well.

Language	internal name
English	english
Brazilian Portuguese	braz_por
French	french
German	german
Polish	polish
Russian	russian
Spanish	spanish
Japanese	japanese
Simplified Chinese	simp_chinese
Korean	korean
Turkish	turkish

Basic example

Localization allows a game object to be rendered with different text depending on certain conditions, typically the user's selected language. In Europa Universalis V, localization connects a localization *key*, which is typically a defined script term such as a country's TAG or the script name of an event, situation, or action, with a defined text string.

Many localization keys have related keys that are generated by the game's engine. For example, TAG_ADJ for a country, or situation_desc for the descriptive text of a situation. Refer to the game's info files or other modding articles on this wiki to determine such keys.

Each localization entry should appear as follows:

Descriptively: each entry is a single line, starting with a single space, the exact localization key, a colon, then another space, and the localized string contained within double quotes. A number can follow the colon, to keep track of revisions, but is not required.

A complete, though short localization file looks like this:

```
l_<lang>:
 key: "Localized text"
```

Comments can be added with # outside quotation marks, on their own line or following a localization entry.

Localization reuse

Localization keys can be used within each other by added `$key$` to the localized string.

For example:

```
key1: "example"
key2: "This is an $key1$"
```

Key2 would appear in game as "This is an example". Reusing localization keys this way is a good way to cut down on required changes in related keys.

Formatting

Similar to other Paradox games, Europa Universalis V has various methods for applying custom formatting to localization. All custom formatting begins with a # followed by the formatting key to begin the formatting. A space is needed after the formatting key to the actual text, but this space will not be represented in the actual text. Finally, to end formatting, a `#!` is necessary. No space should be used before the `#!`, however. Custom formatting can be nested, with each new formatting key requiring its own `#!` in order to end the formatting. Alternatively, custom formatting can be joined with a semi-colon. Text formatting is defined in `/Europa Universalis V/game/<top_folder>/gui/textformatting.gui`. Custom formatting can be defined in any .gui file using `textformatting` blocks, like in the base game file.

Colouring characters

See {{Color}} for using these game text colors on the wiki

An example of colouring some text to red can be seen below:

```
my_loc: "This #R text#! is red!"
```

The following are a sample of formatting keys which are implemented in the base game to colorize text:

#color_red	also #N, #R	Red
#color_green	also #P, #G	Green
#color_concept	also #E	Blue
#color_white	also #V, #Z	White
#color_pure_white		Pure White
#color_yellow		Yellow
#color_black		Black
#color_gray	Also #flavor, #F	Gray
#color_light_gray		Light Gray
#color_dark_gray	Also #ZG	Dark Gray

Other custom formatting

In Europa Universalis V, it is possible to apply other forms of formatting to localization, including bolding and italicizing text. Other forms of formatting to text are similar to colorizing text in that it is necessary to begin the formatting rule with a `#` and end it with a `#!`.

Key	Effect	Example
#!	Ends the current formatting rule.	
#bold	Bolds text.	
#italic	Italicizes text.	

Text icons

It is possible to add icons into localization strings. Base game text icons are defined in the file `/Europa Universalis V/game/<top_folder>/gui/shared/font_icons.gui`, and new icons can be added to any .gui file in `texticon` blocks.

All text icons start with `@` followed by the icon's key and `!` following the key. This has been included in the table below for simplicity.

Note: This is a not a complete list. These are just a few examples of icons.

@warning_icon!	
@gold!	
@legitimacy!	
@trigger_yes!	
@trigger_no!	
@laborers!	
@adm!	

Data functions

Main article: GUI script

Data functions allow for dynamic text to be included in a localization string. Functions are always enclosed in square brackets. Some functions are used alone, while many others can be "dot chained" by combining them with periods. Typically, chained functions are used to shift scopes first then get some localized information from the final scope. Data functions which output a string can also be considered localization functions.

The following are some examples of data functions, many of them start with "Get" and are always written in "UpperCamelCase" or "PascalCase" (e.g. "GetName")

Example data functions

Function name	Description
GetName	Returns the name as a tooltipped string
GetNameWithFlag	Returns a country's name as a tooltipped string with the flag
GetAdjective	Returns a country's adjective as tooltipped string
GetAdjectiveWithNoTooltip	Returns a country's adjective as a plain string
GetAbilityTooltipAsRuler	Returns a character's ability effects if they are a ruler

Many functions require an input argument, such as a type key, variable or saved scope name. These functions use parentheses to enclose the input, with game script objects in single quotes and other data functions without. If multiple input arguments are required, each is separated by a comma.

GetLaw(' ')	Returns the input law as an object for further functions
ScriptValue(' ')	Returns the calculated value of the input named script value

Function formatting

Localization functions can be formatted in the same way as regular localization strings, but they also have special formatting. Function formatting is indicated with a vertical pipe | just before the closing square bracket. All formatting modifiers are put between the pipe and square bracket, and many can be used together.

Format code	Function	Incompatible with
=	Adds "+" to positive values and "-" to negative values	
+	Colors positive values green and negative values red	- Last defined is used
-	Colors positive values red and negative values green	+ Last defined is used
D	Abbreviates integers up to billions (1K, 1M, 1B)	
U	First letter is made uppercase	
%	Converts value to percent (multiply by 100, add percent sign)	
0-5	Determines how many decimal places are shown, max 3 with *	K
<	Unknown at this time	

Concepts

Concepts are built-in info guides, which provide the player info through a tooltip that explains the concept. Concept text is blue by default, but any other formatting applies to the concept text. Concepts are defined in `/Europa Universalis V/game/main_menu/common/game_concepts`. Concepts can be given a custom icon or use a default icon  . Each defined concept then must be localized itself as normal.

To add a concept tooltip to a localization string, use `[<concept_name>]`. Typically concepts are formatted with `|e`. Below is an example of concept added to another localization string:

```
my_loc: "We have no [legitimacy|e]!"
```

Variables

Variables from a script can be passed to a localization string, for instance for custom tooltips:

```
effect = {
	set_local_variable = {
		name = my_variable
		value = 1234.56
	}

	custom_tooltip = {
		text = "my_localization_string"
```

```
l_english:
 my_localization_string: "The value of the variable is [SCOPE.GetLocalVariable('my_variable').GetValue]"
```

Custom tooltips

Localization strings can be defined with custom tooltips as part of the string. To do so the localization string should contain the following:

- `#TOOLTIP:<tooltip_loc_key> <tooltip_text>#!`

This creates a tooltip where the main localization string shows <tooltip_text>, but if that is hovered over, a tooltip shows <tooltip_loc_key>

Saved scopes

Similar to other Paradox games, Europa Universalis V allows for saving game objects for later use. With localization, these saved scopes can be used with localization functions such as "GetName" or "GetHerHis". Depending on the type of saved scope, different functions may or may not be valid and may output different results.

Saved scopes can be included in localization directly or through a localization function. Direct use requires simply using the saved scope by name as if it were a concept, followed by the relevent functions. The localization functions always start with `[SCOPE`, followed by the particular type of scope, and any following functions such as "GetName". In each case, the functions are separated/chained by periods.

Saving a scope is an effect that can be done in most places that effects are valid. Saved scopes can be used for a variety of purposes, including localization. Note however, using a saved scope for localization has some restrictions on timing; typically, the scope must be saved either in an `immediate` block, or in an earlier event or related content object. How long the saved scope persists depends on its use. For example, with situations, the saved scope lasts as long as the situation is active.

Saved scopes use the same data functions as the regular scope type.

Country as saved scope

For this example, the United States of America is saved as an arbitrarily named scope, using the following script:

```
immediate = {
	c:USA = {
		save_scope_as = usa_nation_scope
	}
}
```

Now that the scope for the United States has been saved, it is now possible to use the saved script name in localization functions. This is shown below.

Character as saved scope

For this example, the ruler of a country is saved as an arbitrarily named scope, using the following script:

```
immediate = {
        ruler = {
                save_scope_as = pru_leader
        }
}
```

Now that the character has been saved as a scope, it is now possible to use the saved script name in localization functions.

```
mymod_pru_events.1.f: "Our leader, [pru_leader.GetFullName] has informed us of our magnificence."
mymod_pru_events.1.alt: "Our leader, [SCOPE.sCharacter('pru_leader').GetFullName] has informed us of our
magnificence."
```

Customizable Localization

Unlike other Paradox games, custom localization can only be called with the data function `Custom('custom_localization_name')`. This refers to a scripted function in `/Europa Universalis V/game/in_game/common/customizable_localization`.

Each custom localization has its script name which is referred to with the function, then a type and a list of triggered or randomly selected localization keys. The type determines what scope is used for its triggers, and random_valid determines whether the first valid key or a random valid key is chosen. By default, the first valid key is used.

Example

This is an example of how to create a Customizable Localization file. It should go in the `/common/customizable_localization` folder.

```
ruler_residence = {
    type = country # The scope that the localization applies to. Examples include 'country', 'character' or
'building'
    random_valid = yes # Optional. If no, will pick the first valid option

    text = {
        trigger = { # The conditions that must be met for the key to be valid. The scope is the same as the
localization, in this case, a country.
            government_type = government_type:monarchy
        }
        localization_key = custom_royal_palace
    }

    text = {
        trigger = {
            government_type = government_type:republic
        }
        localization_key = custom_presidential_palace
    }

    text = {
```

```
        }
        localization_key = custom_seaside_palace
    }

    text = {
        trigger = {
            always = no
        }
        fallback = yes # Optional. If fallback is enabled, this option will be picked when there are no other valid options, even if the trigger isn't met.
        localization_key = custom_palace
    }
}
```

To be used, the keys must be defined in a localization file.

```
l_english:
 custom_royal_palace: "royal palace"
 custom_presidential_palace: "presidential palace"
 custom_seaside_palace: "seaside palace"
 custom_palace: "palace"
```

Usage

To use a custom localization, the `Custom()` function must be called in the localization file. Here's how the example custom localization could be used in an event's description:

```
l_english:
 ruler_killed_event.1.t: "Ruler Killed"
 ruler_killer_event.1.d: "Our ruler was killed yesterday, while at the [ROOT.GetCountry.Custom('ruler_residence')]."
```

Depending on the conditions, the code in brackets would be replaced by a randomly selected localization. For instance, if the country is a monarchy and doesn't have any coastal states, it will be replaced by "royal palace". If it's a presidential republic and has coastal states, it will be randomly selected between "presidential palace" or "seaside palace". And if it's neither a monarchy, a presidential republic, nor has any coastal states, it will use the generic fallback "palace".

References

Modding Return to top

Documentation	Defines • Effects • Scopes • Scope links • Triggers | Colors • Macros • Mean time to happen • Modifier types • On actions • Script value • Variables | GUI script • Localization
Scripted content	Actions • Disasters • Events • Missions • Modifiers • Scripted gui • Setup • Situations • Customizable localization
Scripted types	Advances • Art • Buildings • Bureaucracies • Casus belli • Characters • Concepts • Countries • Culture • Diplomacy • Diseases • Estates • Goods • Institutions • International organizations • Laws • Movements • Peace treaties • Pops • Religion • Subject types • Traits • Units • Wargoals
Map	Map • Map modes • Terrain
Graphics	3D Models • Interface • Graphical assets • Fonts • Flags
Audio	Music • Sound
Other	AI • Console commands • Checksum • Mods • Mod compatibility • Mod structure • Troubleshooting
Guides	Interface modding guide • Mod translation • Save-game editing • Settlement position modding guide
Tools	Arcanum • PDX DeepL • PDX Workshop Manager • Community Mod Toolkit • **Add Your Tool to the Wiki**

Sayfa en son 01.13, 27 Mart 2026 tarihinde değiştirildi.